```
# Install required libraries
!pip install transformers datasets accelerate -q
```

```
import torch
import math
from transformers import (
    GPT2LMHeadModel,
    GPT2Tokenizer,
    Trainer,
    TrainingArguments,
    DataCollatorForLanguageModeling,
    set_seed
)
from datasets import Dataset

set_seed(42)
device = "cuda" if torch.cuda.is_available() else "cpu"

print("Using device:", device)
```

```
Using device: cuda
```

```
tokenizer = GPT2Tokenizer.from_pretrained("distilgpt2")   # lighter than GPT2
model = GPT2LMHeadModel.from_pretrained("distilgpt2")

tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id

model.to(device)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set i
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 2.71kB/s]
vocab.json:  1.04M/? [00:00<00:00, 1.74MB/s]
merges.txt:  456k/? [00:00<00:00, 1.00MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to e
tokenizer.json:  1.36M/? [00:00<00:00, 1.84MB/s]
config.json: 100%  762/762 [00:00<00:00, 48.5kB/s]
model.safetensors: 100%  353M/353M [00:02<00:00, 803MB/s]
Loading weights: 100%  76/76 [00:00<00:00, 390.10it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: distilgpt2
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
transformer.h.{0, 1, 2, 3, 4, 5}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100%  124/124 [00:00<00:00, 7.31kB/s]
GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-5): 6 x GPT2Block(
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (attn): GPT2Attention(
          (c_attn): Conv1D(nf=2304, nx=768)
          (c_proj): Conv1D(nf=768, nx=768)
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (mlp): GPT2MLP(
          (c_fc): Conv1D(nf=3072, nx=768)
          (c_proj): Conv1D(nf=768, nx=3072)
          (act): NewGELUActivation()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=768, out_features=50257, bias=False)
)
```

```
def generate_text(model, tokenizer, prompt, max_length=60):
    model.eval()

    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_length=max_length,
            temperature=0.8,
            top_k=50,
            do_sample=True,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
review_prompts = [
    "This product is",
    "I bought this phone and",
    "The quality of this item"
```

```
]

baseline = {}

print("=== BASELINE OUTPUT ===\n")

for prompt in review_prompts:
    text = generate_text(model, tokenizer, prompt)
    baseline[prompt] = text
    print("Prompt:", prompt)
    print("Output:", text)
    print()
```

```
Prompt: The quality of this item
Output: The quality of this item is not affected by the item and not necessarily reflect the quality of this item.
```

```
corpus = [
    "this phone has an amazing battery life and the camera quality is outstanding for the price.",
    "i bought this laptop for college and it handles all my assignments and coding projects perfectly.",
    "the sound quality of these headphones is incredible with deep bass and clear vocals.",
    "this smartwatch tracks my steps accurately and the heart rate monitor is very reliable.",
    "great wireless earbuds with noise cancellation that blocks out all background sound.",
    "the keyboard feels very comfortable for long typing sessions and the backlight is a nice touch.",
    "this portable charger saved me during travel and it charges my phone three times on a single charge.",
    "excellent value for money and the build quality feels premium despite the affordable price.",
```

```
    "best purchase i made this year and i would definitely buy from this brand again."
]
```

```
dataset = Dataset.from_dict({"text": corpus})

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=64
    )

tokenized_dataset = dataset.map(tokenize_function, batched=True)
tokenized_dataset = tokenized_dataset.remove_columns(["text"])

split = tokenized_dataset.train_test_split(test_size=0.2)
```

Map: 100% 9/9 [00:00<00:00, 286.80 examples/s]

```
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False
)
```

```
training_args = TrainingArguments(
    output_dir="./review_model",
    num_train_epochs=5,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    save_strategy="no",
    logging_steps=5,
    learning_rate=5e-5,
    fp16=torch.cuda.is_available()
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=split["train"],
    eval_dataset=split["test"],
    data_collator=data_collator
)
```

```
trainer.train()
```

`loss_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.

[20/20 00:01, Epoch 5/5]

Step	Training Loss
5	4.597963
10	4.492310
15	3.327283
20	3.288662

```
TrainOutput(global_step=20, training_loss=3.9265545845031737, metrics={'train_runtime': 2.8092, 'train_samples_per_second': 12.459, 'train_steps_per_second': 7.119, 'total_flos': 571586641920.0, 'train_loss': 3.9265545845031737, 'epoch': 5.0})
```

```
eval_result = trainer.evaluate()
print("Perplexity:", math.exp(eval_result["eval_loss"]))
```

[1/1 : < :]

```
Perplexity: 37.0441739703586
```

```
print("\n=== AFTER FINE-TUNING ===\n")

for prompt in review_prompts:
    output = generate_text(model, tokenizer, prompt)
    print("Prompt:", prompt)
    print("Baseline:", baseline[prompt])
    print("Fine-Tuned:", output)
```

```
print()
```

```
Fine-Tuned: I bought this phone and my husband was so impressed with it, I took it in my pocket and it took me out on my way ho

Prompt: The quality of this item
Baseline: The quality of this item is not affected by the item and not necessarily reflect the quality of this item.
```

```
Fine-Tuned: The quality of this item is good and the price I am willing to pay for it. The quality of this item is fantastic. T
```

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer

tokenizer2 = GPT2Tokenizer.from_pretrained("distilgpt2")
model2 = GPT2LMHeadModel.from_pretrained("distilgpt2")

tokenizer2.pad_token = tokenizer2.eos_token
model2.config.pad_token_id = tokenizer2.eos_token_id

model2.to(device)
```

```
Loading weights: 100%                                    76/76 [00:00<00:00, 309.87it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: distilgpt2
Key                                        | Status     |  |
-------------------------------------------+------------+--+-
transformer.h.{0, 1, 2, 3, 4, 5}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 768)
    (wpe): Embedding(1024, 768)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-5): 6 x GPT2Block(
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (attn): GPT2Attention(
          (c_attn): Conv1D(nf=2304, nx=768)
          (c_proj): Conv1D(nf=768, nx=768)
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (mlp): GPT2MLP(
          (c_fc): Conv1D(nf=3072, nx=768)
          (c_proj): Conv1D(nf=768, nx=3072)
          (act): NewGELUActivation()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=768, out_features=50257, bias=False)
)
```

```
recipe_prompts = [
    "To make butter chicken",
    "For pasta carbonara",
    "To prepare chocolate cake"
]

baseline2 = {}

print("=== BASELINE RECIPES ===\n")

for prompt in recipe_prompts:
    text = generate_text(model2, tokenizer2, prompt)
    baseline2[prompt] = text
    print("Prompt:", prompt)
    print("Output:", text)
    print()
```

```
=== BASELINE RECIPES ===

Prompt: To make butter chicken
Output: To make butter chicken. You will need 1/2 lb of butter chicken chicken or 1/2 cup of butter chicken.

Prompt: For pasta carbonara
Output: For pasta carbonara or spaghetti. This is something that is often overlooked by most Italian pasta lovers.
If you are looking for an Italian pasta with a very light texture, you need to be able to place it in a cup of water or a warm h


Prompt: To prepare chocolate cake
Output: To prepare chocolate cake.

But since I had previously eaten these cookies, I really, really needed that one because I loved them so much.
This dessert, which I have a few more years to eat (and I am going to spoil a bit later on).
So I have decided
```

```
recipes = [
    "to make butter chicken start by marinating chicken pieces in yogurt with turmeric chili powder and garam masala for one hc
    "heat butter in a pan and fry onions until golden brown then add ginger garlic paste and cook for two minutes.",
    "add tomato puree and cook on low heat for ten minutes until the oil separates from the masala.",
    "add the marinated chicken and cook on medium heat for fifteen minutes until fully cooked.",
    "finish with fresh cream and kasuri methi and serve hot with naan or steamed rice.",
```

```
    "for pasta carbonara boil spaghetti in salted water until al dente and reserve half cup of pasta water.",
    "fry diced pancetta in olive oil until crispy and set aside.",
    "whisk together egg yolks parmesan cheese and black pepper in a bowl.",
    "toss the hot pasta with pancetta and remove from heat then quickly stir in the egg mixture.",
    "the residual heat will cook the eggs into a creamy sauce and serve immediately with extra parmesan.",

    "for chocolate chip cookies cream together butter and sugar until light and fluffy.",
    "beat in eggs one at a time then add vanilla extract and mix well.",
    "fold in flour baking soda and salt then gently stir in chocolate chips.",
    "scoop dough onto a baking sheet and bake at 180 degrees for twelve minutes until golden.",
    "let cookies cool on the tray for five minutes before transferring to a wire rack."
]
```

```
dataset2 = Dataset.from_dict({"text": recipes})

def tokenize_recipe(examples):
    return tokenizer2(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=64
    )

tokenized_dataset2 = dataset2.map(tokenize_recipe, batched=True)
tokenized_dataset2 = tokenized_dataset2.remove_columns(["text"])

split2 = tokenized_dataset2.train_test_split(test_size=0.2)
```

Map: 100% 15/15 [00:00<00:00, 304.08 examples/s]

```
collator2 = DataCollatorForLanguageModeling(
    tokenizer=tokenizer2,
    mlm=False
)
```

```
training_args2 = TrainingArguments(
    output_dir="./recipe_model",
    num_train_epochs=5,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    save_strategy="no",
    logging_steps=5,
    learning_rate=5e-5,
    fp16=torch.cuda.is_available()
)
```

```
trainer2 = Trainer(
    model=model2,
    args=training_args2,
    train_dataset=split2["train"],
    eval_dataset=split2["test"],
    data_collator=collator2
)
```

```
trainer2.train()
```

[30/30 00:02, Epoch 5/5]

Step	Training Loss
5	4.177651
10	4.025881
15	3.262423
20	2.710777
25	2.889902
30	2.623104

```
TrainOutput(global_step=30, training_loss=3.281622854868571, metrics={'train_runtime': 2.8816, 'train_samples_per_second':
20.822, 'train_steps_per_second': 10.411, 'total_flos': 979862814720.0, 'train_loss': 3.281622854868571, 'epoch': 5.0})
```

```
eval_result2 = trainer2.evaluate()
print("Perplexity:", math.exp(eval_result2["eval_loss"]))
```